MODULE

12

Advanced OOP

Classes that feel built-in, clean designs, and the OOP questions every interview asks.

YOU WILL LEARN

Dunder methods	01_dunder_methods.py
Properties	02_properties.py
class & static methods	03_class_and_static_methods.py
Inheritance, super(), MRO	04_inheritance_and_mro.py
ABCs & Protocols	05_abc_and_protocols.py
Composition & SOLID	06_composition_and_solid.py
Descriptors, __new__, metaclasses	07_descriptors_and_metaclasses.py
Interview Q&A	INTERVIEW.md

WHY DUNDER METHODS

Make your objects feel built-in

THE PROBLEM

You built a **Money** class. Now **print(price)** shows **<Money object at 0x7f...>**, **price1 + price2** crashes, and **==** says two equal amounts are different.

THE IDEA

Python calls special __dunder__ methods behind the scenes. Define them and +, ==, print, len, for just work.

STEP BY STEP DUNDER METHODS

You write this, Python calls that

YOU WRITE	PYTHON CALLS	TYPICAL USE
`print(obj), f"{obj}"`	`__str__`	friendly text for users
`obj in a shell / logs`	`__repr__`	unambiguous text for developers
`a + b, a * 3`	`__add__`, `__mul__`	Money, Vector, Duration
`a == b, a < b`	`__eq__`, `__lt__`	compare and sort
`len(obj), obj[i], for x in obj`	`__len__`, `__getitem__`, `__iter__`	act like a container
`with obj:`	`__enter__`, `__exit__`	clean-up (Module 13)

TRY IT DUNDER METHODS

A Money class that behaves

```
class Money:
    def __init__(self, cents, currency="EUR"):
        self.cents, self.currency = cents, currency

    def __repr__(self):
        return f"Money({self.cents}, {self.currency!r})"

    def __str__(self):
        return f"{self.cents / 100:.2f} {self.currency}"

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError("currency mismatch")
        return Money(self.cents + other.cents, self.currency)

    def __eq__(self, other):
        return (self.cents, self.currency) == (other.cents, other.currency)

a, b = Money(1999), Money(500)
print(a + b, "|", repr(a + b), "|", a == Money(1999))
print(sum([a, b], Money(0)))
```

OUTPUT

```
24.99 EUR | Money(2499, 'EUR') | True
24.99 EUR
```

1. **__repr__** for developers, **__str__** for users.
2. Store money as **integer cents** (or Decimal), never float.
3. **sum()** works because **__add__** exists.

TRY IT PROPERTIES

Attributes with rules

```python
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance        # goes through the setter!

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("balance can't be negative")
        self._balance = value

acc = Account("vishal", 100)
acc.balance += 50
print(acc.balance)
try:
    acc.balance = -5
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
150
ValueError: balance can't be negative
```

1. Callers still write **acc.balance**: no getters and setters.
2. The rule lives in **one** place.
3. **_balance** with one underscore = ”internal, please don’t touch”.

STEP BY STEP METHODS

Instance vs class vs static methods

	FIRST ARGUMENT	CAN USE	TYPICAL USE
`instance method`	`self`	this object's data	almost everything
`@classmethod`	`cls`	the class (works for subclasses)	**alternative constructors**: `User.from_json(...)`
`@staticmethod`	nothing	neither	a helper that belongs to the topic

> Interview one-liner: **classmethod** gets the class, **staticmethod** gets nothing, a normal method gets the instance.

TRY IT METHODS

classmethod as a second constructor

```python
import json

class User:
    def __init__(self, name, email):
        self.name, self.email = name, email

    @classmethod
    def from_json(cls, text):
        data = json.loads(text)
        return cls(data["name"], data["email"])    # cls, not User

    @staticmethod
    def is_valid_email(email):
        return "@" in email and "." in email.split("@")[-1]

class Admin(User):
    pass

a = Admin.from_json('{"name": "vishal", "email": "v@ex.com"}')
print(type(a).__name__, a.name, User.is_valid_email(a.email))
```

OUTPUT

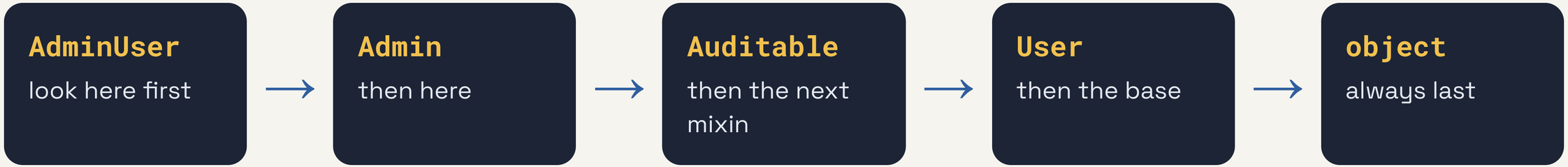
```
Admin vishal True
```

1. **cls(...)** builds the **right** class: Admin.from_json gives an Admin.
2. Static methods are plain functions grouped inside the class.

PICTURE IT INHERITANCE AND MRO

Who answers a method call?

🧭 Python searches classes in a fixed order called the **MRO** (Method Resolution Order). **super()** means "the **next** class in the MRO", not just "my parent".



See it with **AdminUser.__mro__**. The diamond problem is solved by this one fixed order.

TRY IT INHERITANCE AND MRO

super() follows the MRO

```python
class User:
    def describe(self):
        return ["user"]

class Admin(User):
    def describe(self):
        return ["admin"] + super().describe()

class Auditable(User):
    def describe(self):
        return ["audited"] + super().describe()

class AdminUser(Admin, Auditable):
    pass

print([c.__name__ for c in AdminUser.__mro__])
print(AdminUser().describe())
```

OUTPUT

```
['AdminUser', 'Admin', 'Auditable', 'User', 'object']
['admin', 'audited', 'user']
```

1. Admin's **super()** goes to **Auditable**, not User!
2. Every class is visited **once**.
3. Prefer shallow hierarchies; use mixins for small add-ons.

TRY IT ABCS AND PROTOCOLS

Define a contract

```python
from abc import ABC, abstractmethod
from typing import Protocol

class PaymentGateway(ABC):                  # nominal: must inherit
    @abstractmethod
    def charge(self, cents: int) -> str: ...

class Stripe(PaymentGateway):
    def charge(self, cents): return f"stripe charged {cents}"

class Notifier(Protocol):                   # structural: just match
    def send(self, msg: str) -> None: ...

class Slack:                                # no inheritance needed
    def send(self, msg): print("slack:", msg)

def alert(n: Notifier): n.send("disk full")
alert(Slack())
print(Stripe().charge(500))
```

OUTPUT

```
slack: disk full
stripe charged 500
```

1. **ABC**: subclasses **must** implement the abstract methods.
2. **Protocol**: "anything with a .send()" (duck typing, checked by mypy).
3. Code depends on the contract, so tests can pass in fakes.

COMMON MISTAKE COMPOSITION

Inheritance for code reuse

✗ WRONG

```
class Database: ...
class EmailSender: ...

class OrderService(Database, EmailSender):
    ...   # an OrderService IS a Database??
```

Inheriting just to **reuse** code creates weird ”is-a” relationships, deep hierarchies and classes that are hard to test.

✓ RIGHT

```
class OrderService:
    def __init__(self, db, mailer):
        self.db = db            # HAS a db
        self.mailer = mailer    # HAS a mailer

service = OrderService(FakeDB(), FakeMailer())
```

Composition: pass collaborators in (dependency injection). Swap real ones for fakes in tests.

STEP BY STEP SOLID

SOLID in plain words

LETTER	PRINCIPLE	IN ONE SENTENCE
S	Single responsibility	a class has one reason to change
O	Open / closed	add features by adding code, not editing old code
L	Liskov substitution	a subclass must work wherever the parent works
I	Interface segregation	small, focused interfaces (Protocols)
D	Dependency inversion	depend on abstractions, inject the details

> You won't recite these at work, but interviews love them. Give one example for each.

TRY IT UNDER THE HOOD

__new__ and __init_subclass__: a plugin registry

```
class Exporter:
    registry = {}

    def __init_subclass__(cls, fmt, **kw):
        super().__init_subclass__(**kw)
        Exporter.registry[fmt] = cls        # auto-register

class CSVExporter(Exporter, fmt="csv"):
    def run(self): return "a,b"

class JSONExporter(Exporter, fmt="json"):
    def run(self): return "{}"

print(Exporter.registry)
print(Exporter.registry["json"]().run())
```

OUTPUT

```
{'csv': <class '__main__.CSVExporter'>,
'json': <class '__main__.JSONExporter'>}
{}
```

1. **__init_subclass__** runs when a subclass is **defined**.
2. **__new__ creates** the object; **__init__** fills it in.
3. Metaclasses can do this too, but are rarely needed.

IN REAL JOBS OOP

OOP in real web code

Django models and DRF serializers

```
class Order(models.Model):
    def __str__(self): return f"#{self.pk}"
```

Pydantic models for API bodies

```
class OrderIn(BaseModel):
    @classmethod ...   # validators
```

Service layer with injected repos

```
class OrderService:
    def __init__(self, repo: OrderRepo): ...
```

Swappable integrations

```
class Notifier(Protocol):
    def send(self, msg: str) -> None: ...
```

INTERVIEW SAY THIS IN THE INTERVIEW

“@classmethod vs @staticmethod vs instance method?”

- An **instance method** gets **self**: it works with one object’s data.
- A **classmethod** gets **cls**: used for **alternative constructors** (**from_json**, **from_row**), and it respects subclasses.
- A **staticmethod** gets nothing: a utility function that belongs to the class’s topic.
- Example: **datetime.fromtimestamp()** and **dict.fromkeys()** are classmethods.

Follow-up they ask next: ”What is the MRO? How does super() work with multiple inheritance?” (C3 linearisation; super() calls the next class in the MRO, so each class runs once).

INTERVIEW SAY THIS IN THE INTERVIEW

“Composition vs inheritance: which do you prefer?”

- **Inheritance** models ”**is-a**” (an Admin is a User) and is good for small, stable hierarchies.
- **Composition** models ”**has-a**” (a service has a repository) and is more flexible.
- I prefer **composition + dependency injection**: parts are swappable and easy to test with fakes.
- I use ABCs or Protocols to define the contract between the parts.

```
class OrderService:
    def __init__(self, repo: OrderRepo,
                 payments: PaymentGateway):
        self.repo = repo
        self.payments = payments
```

Follow-up they ask next: ”Explain the SOLID principles” and ”What design patterns have you used?” (factory, strategy, repository, singleton via a module, observer via signals).

RECAP REMEMBER THESE

Module 12 in 6 lines

1. **__repr__**, **__str__**, **__eq__**, **__add__**… make objects feel built-in.
2. **@property** adds rules without changing how callers use the attribute.
3. classmethod = **cls** (constructors); staticmethod = nothing; method = **self**.
4. **super()** = the next class in the **MRO**.
5. ABC = must inherit; Protocol = just match the shape.
6. Prefer **composition** and inject dependencies.

PRACTICE NOW

Do exercises/ex12_vector.py: a Vector class with +, *, ==, abs() and len().